6.3 A specialized product controller

Product pages belong together. Add a Product data class and a ProductController rather than putting product-specific work into Application or DefaultController. This step prepares the PHP classes; 6.4 adds the view and route before we request the new page.

Create the data class

Create `src/Tutorials/Class6/Product.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

use InvalidArgumentException;

class Product
{
    public string $name {
        set {
            $value = trim($value);
            if ($value === '') {
                throw new InvalidArgumentException('Product name must not be
empty.');
            }
            $this->name = $value;
        }
    }

    public float $price {
        set {
            if (!is_finite($value) || $value <= 0) {
                throw new InvalidArgumentException('Product price must be finite
and greater than zero.');
            }
            $this->price = $value;
        }
    }

    public function __construct(string $name, float $price)
    {
        $this->name = $name;
        $this->price = $price;
    }
}
```

Each Product holds one name and price. We extend the chapter's public typed properties with **PHP 8.4 property hooks**. Use PHP 8.4 or newer for both the command line and web server; earlier versions cannot

parse this syntax. A Product represents data; it neither reads requests nor renders HTML. The float price is sufficient for this display example; no payment calculations are involved.

Both properties allow public reads and writes. The set hook intercepts assignments: name trims surrounding whitespace and rejects an empty result; price rejects zero, negative values, and non-finite values. Throwing InvalidArgumentException stops the assignment before the stored value changes. Assignments inside the constructor also invoke these hooks, so initial values and later changes obey the same rules.

These are backed properties: assigning `$this->name` inside its own set hook stores the validated value without recursively calling that hook. We omit get hooks because normal public reads already return the stored values. Twig can still read `product.name` and `product.price`. See the PHP property hooks manual for backed-property behavior.

Create the page controller

Create `src/Tutorials/Class6/ProductController.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

class ProductController
{
    private const PATH_TO_TEMPLATES = __DIR__ .
'/../../../templates/tutorials/class6';

    private Environment $twig;

    public function __construct()
    {
        $loader = new FilesystemLoader(self::PATH_TO_TEMPLATES);
        $this->twig = new Environment($loader, [
            'autoescape' => 'html',
            'strict_variables' => true,
        ]);
    }

    public function productList(): void
    {
        $products = [
            new Product('Study notebook', 4.50),
            new Product('Desk organizer', 12.00),
            new Product('Pencil set', 3.25),
        ];

        $template = 'products.html.twig';
        $args = ['products' => $products];
        echo $this->twig->render($template, $args);
```

```
    }
}
```

productList builds an array of three objects, puts it under the products context key, and passes that context to Twig. Product and ProductController share the same namespace, so they need no use statement for one another. Composer loads their class files through the existing App mapping.

We intentionally repeat the Twig setup from DefaultController at this stage, following the chapter. This makes the shared code visible before we extract it in 6.5. Do not add inheritance yet.

Check the preparation

From the student repository root, check both files:

```
php -l src/Tutorials/Class6/Product.php
php -l src/Tutorials/Class6/ProductController.php
```

Home and Contact still work. Products is not wired into Application yet, so `?action=products` still selects the Home fallback. Calling productList directly now would fail because its template does not exist until 6.4. Syntax checks verify these new class definitions; the next tutorial verifies the complete feature.

Instructor prompt: “Which object represents one supply item, and which object prepares the list page?” Then identify the duplicated constant, property, and constructor in the two page controllers.